EXPERIMENT NO. 10

Sort an Array in O(n log n) Time with Minimum Space

Programming Language: Java

Name : A Yuvan Charan
Register No. : 192424383

Aim

To write a Java program that sorts an array in ascending order without built-in functions in O(n log n) time and the smallest possible space (heap sort).

Question

Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(n log n) time complexity and with the smallest space complexity possible.

Example 1

Input: nums = [5,2,3,1]

Output: [1,2,3,5]

Example 2

Input: nums = [5,1,1,2,0,0]

Output: [0,0,1,1,2,5]

Algorithm

Step 1: Start.

Step 2: Read the array nums of size n.

Step 3: Build a max-heap: for i = n/2 - 1 down to 0, call heapify(nums, n, i).

Step 4: For i = n-1 down to 1, swap nums[0] (the largest) with nums[i], then call heapify(nums, i, 0) to restore the heap on the remaining elements.

Step 5: heapify(i): compare nums[i] with its children 2i+1 and 2i+2, swap with the larger child if needed and continue downward.

Step 6: Print the sorted array.

Step 7: Stop.

Program (Java)

```
public class Main {

    static void heapify(int[] nums, int n, int i) {
        while (true) {
            int largest = i, left = 2 * i + 1, right = 2 * i + 2;
            if (left < n && nums[left] > nums[largest])
```

```
                largest = left;
            if (right < n && nums[right] > nums[largest])
                largest = right;
            if (largest == i)
                return;
            int t = nums[i];
            nums[i] = nums[largest];
            nums[largest] = t;
            i = largest;
        }
    }

    static void heapSort(int[] nums) {
        int n = nums.length;
        for (int i = n / 2 - 1; i >= 0; i--)      // build max-heap
            heapify(nums, n, i);
        for (int i = n - 1; i > 0; i--) {         // extract maximum
            int t = nums[0];
            nums[0] = nums[i];
            nums[i] = t;
            heapify(nums, i, 0);
        }
    }

    static void printArray(int[] nums) {
        StringBuilder sb = new StringBuilder("Output: [");
        for (int i = 0; i < nums.length; i++)
            sb.append(i > 0 ? "," : "").append(nums[i]);
        System.out.println(sb.append("]"));
    }

    public static void main(String[] args) {
        int[] a = {5, 2, 3, 1};
        int[] b = {5, 1, 1, 2, 0, 0};

        heapSort(a);
        printArray(a);
        heapSort(b);
        printArray(b);
    }
}
```

Output

```
Output: [1,2,3,5]
Output: [0,0,1,1,2,5]

=== Code Execution Successful ===
```

Complexity Analysis

Time Complexity: O(n log n) in the best, average and worst case.

Space Complexity: O(1) - the array is sorted in place and no extra array is used (the heapify loop is iterative).

Result

Thus the program was executed successfully and the output was verified.